

Vettori (vector)

Vettori (vector)

Come usare il contenitore vector della STL in C++.

Cos'è un vector?

Un array normale ha un problema fondamentale: la dimensione deve essere decisa prima di eseguire il programma. Se non sai in anticipo quanti dati avrai, sei bloccato.

Il **vector** risolve questo problema: è come un array, ma **cambia dimensione automaticamente** mentre il programma gira. Aggiungi elementi e lui cresce. Rimuovili e lui si riduce.

Per usarlo, includi `<vector>` :

```
#include <vector>
using namespace std;
```

Creare un vector

```
// Vector vuoto – aggiungeremo elementi dopo
vector<int> v1;

// Vector con 5 elementi, tutti a zero
vector<int> v2(5);           // {0, 0, 0, 0, 0}

// Vector con valori già impostati
vector<int> v3 = {1, 2, 3, 4, 5};

// Vector con 4 elementi, tutti uguali a 7
vector<int> v4(4, 7);        // {7, 7, 7, 7}

// Vector di stringhe
vector<string> nomi = {"Alice", "Bob", "Carlo"};
```

Il tipo tra `<>` indica cosa contiene il vector. `vector<int>` contiene interi, `vector<string>` contiene stringhe, e così via.

Accedere agli elementi

Come gli array, si usa l'indice `[]` (parte da 0):

```
vector<int> v = {10, 20, 30, 40, 50};

cout << v[0] << endl;    // 10 – primo elemento
cout << v[2] << endl;    // 30
cout << v[4] << endl;    // 50 – ultimo elemento

// Modifica di un elemento
v[1] = 99;
cout << v[1] << endl;    // 99
```

Per un accesso più sicuro (il programma ti avvisa se esci dai limiti), usa `.at()` :

```
cout << v.at(2) << endl;    // 30
// v.at(10);                // lancia un errore se 10 è fuori range
```

Aggiungere e rimuovere elementi

```
vector<int> v;

// Aggiunge in coda – il vector cresce automaticamente
v.push_back(10);
v.push_back(20);
v.push_back(30);
// v è ora {10, 20, 30}

// Rimuove l'ultimo elemento
v.pop_back();
// v è ora {10, 20}
```

Informazioni sul vector

```
vector<int> v = {1, 2, 3, 4, 5};

cout << v.size() << endl;    // 5 – quanti elementi contiene
cout << v.empty() << endl;   // false (0) – è vuoto?

v.pop_back();
cout << v.size() << endl;    // 4

v.clear();                   // svuota completamente il vector
cout << v.size() << endl;    // 0
cout << v.empty() << endl;   // true (1)
```

Scorrere un vector

Con indice (come un array):

```
vector<int> v = {10, 20, 30, 40, 50};

for (int i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
}
```

Con range-based for (il modo preferito — più semplice):

```
for (int n : v) {
    cout << n << " ";
}
```

Per modificare gli elementi mentre scorri, usa `&` :

```
// Moltiplica ogni elemento per 2
for (int& n : v) {
    n *= 2;
}
```

Primo e ultimo elemento

```
vector<int> v = {10, 20, 30};

cout << v.front() << endl;    // 10 — primo elemento
cout << v.back() << endl;     // 30 — ultimo elemento
```

Inserire e rimuovere in posizioni specifiche

```
vector<int> v = {1, 2, 4, 5};

// Inserisce 3 alla posizione 2 (tra il 2 e il 4)
v.insert(v.begin() + 2, 3);
// v è ora {1, 2, 3, 4, 5}

// Rimuove l'elemento alla posizione 1 (il 2)
v.erase(v.begin() + 1);
// v è ora {1, 3, 4, 5}
```

Ordinare un vector

Per ordinare, includi anche `<algorithm>` :

```
#include <algorithm>

vector<int> v = {5, 2, 8, 1, 9, 3};

sort(v.begin(), v.end());           // ordine crescente → {1, 2, 3, 5, 8, 9}
sort(v.begin(), v.end(), greater<int>()); // ordine decrescente → {9, 8, 5, 3, 2, 1}
```

Esempio pratico: raccolta voti interattiva

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> voti;
    int voto;

    cout << "Inserisci i voti (0 per terminare):" << endl;
    while (true) {
        cin >> voto;
        if (voto == 0) break;

        if (voto >= 1 && voto <= 10) {
            voti.push_back(voto); // aggiungiamo il voto al vector
        } else {
            cout << "Voto non valido, ignorato." << endl;
        }
    }

    if (voti.empty()) {
        cout << "Nessun voto inserito." << endl;
        return 0;
    }

    // Calcoliamo le statistiche
    int somma = 0;
    for (int v : voti) somma += v;
    double media = (double)somma / voti.size();

    // Ordiniamo per trovare min e max
    sort(voti.begin(), voti.end());

    cout << "\nNumero di voti: " << voti.size() << endl;
    cout << "Media: " << media << endl;
    cout << "Minimo: " << voti.front() << endl; // primo dopo ordinamento
    cout << "Massimo: " << voti.back() << endl; // ultimo dopo
ordinamento

    cout << "Voti ordinati: ";
    for (int v : voti) cout << v << " ";
    cout << endl;
}
```

```
}    return 0;
```